

L2 - 13.2. x86 Assembly Crash Course

Task 1 - Introduction

Assembly is the most reliable language for reverse engineering compiled binaries. Compilation removes high-level code details, making assembly the best source for malware analysis.

Task 2 - Opcodes and Operands

Machine code consists of hexadecimal opcodes and operands that represent CPU instructions.

Opcodes: An **opcode (operation code)** is the part of a machine instruction that tells the **CPU what operation to perform**. Examples of operations:

- `mov` → Move data
- `add` → Add values
- `sub` → Subtract values
- `cmp` → Compare values
- `jmp` → Jump to another instruction

Example: `mov eax, 5` [Opcode: `mov` (move data)]

Operands: An **operand** is the **data or location** that the opcode operates on. Operands can be:

- **Registers** (e.g., `eax`, `ebx`)
- **Immediate values** (constants like `5` or `0x5F`)
- **Memory addresses** (e.g., `[eax]`, `[0x401000]`)

Types of Operands

In general, there are three types of operands in the assembly language.

- Immediate Operands can also be considered constants. These are fixed values like we had the `0x5f` in the above example.
- Registers can also be operands. The above example shows `eax` as a register where the immediate operand is stored.
- Memory operands are denoted by square brackets, and they reference memory locations. For example, if we see `[eax]` as an operand, it will mean that the value in `eax` is the memory location on which the operation has to be performed.

`add eax, ebx` [Opcode: `add` → Perform addition.]

Operands:

`eax` → Destination register.

`ebx` → Source register.

Result: `eax = eax + ebx`

Machine Code Example

`040000: b8 5f 00 00 00 mov eax, 0x5f`

`040000` → Memory address of the instruction.

`b8` → Opcode (represents `mov eax`).

`5f 00 00 00` → Operand (the value `0x5F`, stored in little-endian format).

- **Opcode:** Specifies **what action** the CPU should perform.
- **Operand:** Specifies **what data or location** the action is performed on.

Answer the questions below

| What are the hex codes that denote the assembly operations called?

Opcodes

| Which type of operand is denoted by square brackets? memory

operand

Task 3 - General Instructions

These instructions perform simple operations, such as moving a value from one type of storage to another.

The MOV Instruction: The `mov` instruction copies data from a **source** to a **destination**. It can move values between registers, memory locations, and immediate values, making it the most common instruction for transferring data.

Memory Addressing with MOV: Square brackets (`[]`) indicate a **memory address**. The CPU retrieves the value stored at that address. Offsets, such as `[ebp+4]`, are used to access nearby memory locations like local variables or function arguments.

The LEA Instruction: The `lea` (Load Effective Address) instruction loads the **address** of a memory location rather than the value stored there. It is commonly used for pointer and address calculations.

The NOP Instruction: The `nop` (No Operation) instruction performs no action. It is mainly used for code alignment, padding, and creating **NOP sleds** in malware to help redirect execution safely.

Shift Instructions: Shift instructions move bits left (`shl`) or right (`shr`). Left shifts multiply a value by powers of two, while right shifts divide it. The last bit shifted out is stored in the **Carry Flag (CF)**.

Using Shifts for Arithmetic: Shifts provide a fast way to multiply or divide by powers of two, making them more efficient than standard arithmetic instructions.

Rotate Instructions: Rotate instructions (`rol` and `ror`) move bits in a circular manner, returning shifted-out bits to the opposite end of the register. Unlike shifts, no bits are lost during the operation.

Answer the questions below

| In `mov eax, ebx`, which register is the destination operand?

eax

| What instruction performs no action?

nop

Task 4 - Flags

In x86 assembly language, the CPU has several flags that indicate the outcome of certain operations or conditions. These flags are bits in a special register known

as the **flags register** or **EFLAGS** register.

Flag	Abbreviation	Explanation
Carry	CF	Set when a carry-out or borrow is required from the most significant bit in an arithmetic operation. Also used for bit-wise shifting operations.
Parity	PF	Set if the least significant byte of the result contains an even number of 1 bits.
Auxiliary	AF	Set if a carry-out or borrow is required from bit 3 to bit 4 in an arithmetic operation (BCD arithmetic).
Zero	ZF	Set if the result of the operation is zero.
Sign	SF	Set if the result of the operation is negative (i.e., the most significant bit is 1).
Overflow	OF	Set if there's a signed arithmetic overflow (e.g., adding two positive numbers and getting a negative result or vice versa).
Direction	DF	Determines the direction for string processing instructions. If DF=0, the string is processed forward; if DF=1, the string is processed backward.
Interrupt Enable	IF	If set (1), it enables maskable hardware interrupts. If cleared (0), interrupts are disabled.

Answer the questions below

| Which flag will be set if the result of the operation is zero?

ZF

| Which flag will be set if the result of the operation is negative?

SF

Task 5 - Arithmetic and Logical Instructions

Arithmetic Operations: Arithmetic Operations are performed by a CPU using arithmetic instructions.

Addition and Subtraction Instructions: The `add` and `sub` instructions perform basic arithmetic. `add` adds a value to the destination, while `sub` subtracts a value

from it. The result is stored in the destination register, and the CPU updates flags such as the **Zero Flag (ZF)** and **Carry Flag (CF)**.

Multiplication and Division Instructions: The `mul` and `div` instructions use the **EAX** and **EDX** registers. `mul` multiplies the value in `EAX` and stores the result across `EDX:EAX`, while `div` divides the value in `EDX:EAX`, storing the quotient in `EAX` and the remainder in `EDX`.

Increment and Decrement Instructions: The `inc` and `dec` instructions increase or decrease a register's value by **1**. They are commonly used in loops and counters.

Logical Instructions: Logical instructions are used to perform logical operations.

AND Instruction: The `and` instruction performs a **bitwise AND** operation. A bit is set to **1 only if both corresponding bits are 1**, making it useful for masking or clearing specific bits.

OR Instruction: The `or` instruction performs a **bitwise OR** operation. A bit is set to **1 if either operand contains a 1**, making it useful for setting specific bits.

NOT Instruction: The `not` instruction **inverts all bits** in the operand, changing every **1 to 0** and every **0 to 1**.

XOR Instruction: The `xor` instruction performs a **bitwise XOR** operation. A bit is set to **1 only if the two bits are different**. It is commonly used to **clear a register** by XORing it with itself (`xor eax, eax`), which sets the register to zero efficiently.

Answer the questions below

In a subtraction operation, which flag is set if the destination is smaller than the subtracted value?

Carry Flag

Which instruction is used to increase the value of a register?

`inc`

Do the following instructions have the same result?

`(xor eax, eax)` `mov eax, 0`

Task 6 - Conditionals and Branching

Conditionals: The CPU uses **conditional instructions** to compare values (equal, greater than, or less than). These instructions update **flags** that determine what the program should do next.

The TEST Instruction: The `TEST` instruction performs a **bitwise AND** between two operands but **does not save the result**. Instead, it updates the **Zero Flag (ZF)**. It is commonly used to quickly check whether a value is **0 (NULL)**. `test destination, source`

The CMP Instruction: The `CMP` instruction **compares two values** by subtracting them internally without changing either value. It updates flags such as:

- **ZF = 1** if both values are equal.
- **CF = 1** if the source is greater than the destination (unsigned comparison).
- Both flags are cleared if the destination is greater.

`cmp destination, source`

Branching: Normally, the CPU executes instructions **one after another**. **Branching** changes this sequence by updating the **Instruction Pointer (IP/EIP/RIP)**, allowing the program to jump to another location.

The JMP Instruction: The `JMP` instruction performs an **unconditional jump** to another address. The CPU immediately continues execution from the specified location. `jmp location`

Conditional Jumps: Conditional jumps perform a jump **only if a specific condition is true**, based on the CPU's **flag registers** (such as ZF and CF). They are the assembly equivalent of **if statements** in high-level languages.

Instruction	Explanation
jz	Jump if the ZF is set (ZF=1).
jnz	Jump if the ZF is not set (ZF=0).
je	Jump if equal. Often used after a CMP instruction.
jne	Jump if not equal. Often used after a CMP instruction.
jg	Jump if the destination is greater than the source operand. Performs signed comparison and is often used after a CMP instruction.

jl	Jump if the destination is lesser than the source operand. Performs signed comparison and is often used after a CMP instruction.
jge	Jump if greater than or equal to. Jumps if the destination operand is greater than or equal to the source operand. Similar to the above instructions.
jle	Jump if lesser than or equal to. Jumps if the destination operand is lesser than or equal to the source operand. Similar to the above instructions.
ja	Jump if above. Similar to jg, but performs an unsigned comparison.
jb	Jump if below. Similar to jl, but performs an unsigned comparison.
jae	Jump if above or equal to. Similar to the above instructions.
jbe	Jump if below or equal to. Similar to the above instructions.

Answer the questions below

Which flag is set as a result of the test instruction being zero?

Zero flag

Which of the below operations uses subtraction to test two values?

1

1. cmp eax, ebx
2. test eax, ebx

Which flag is used to identify whether a jump will be taken or not after a jz or jnz instruction?

Zero Flag

Task 7 - Stack and Function calls

The Stack: The stack is a **Last In, First Out (LIFO)** memory area. The last value added to the stack is the first one removed. It is mainly used for storing local variables, function arguments, and return addresses during program execution.

The PUSH Instruction: The `push` instruction adds a value to the top of the stack. It stores the value at the memory location pointed to by the **Stack Pointer (ESP)** and

then decreases ESP to point to the new top of the stack.

pusha (push all words): Pushes all the 16-bit general purpose registers to the stack, i.e. AX, BX, CX, DX, SI, DI, SP, BP

pushad (push all double words): Pushes all the 32-bit general purpose registers to the stack, i.e. EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP

The POP Instruction: The `pop` instruction removes the value from the top of the stack and places it into a destination register or memory location. After removing the value, the **Stack Pointer (ESP)** is increased to point to the next stack item.

popa (pop all words): Pops the values sequentially from the top of the stack to general-purpose registers in the following order: DI, SI, BP, BX, DX, CX, AX. The SP or ESP is adjusted to reflect the new top position of the stack.

popad (pop all double words): Pops the values sequentially from the top of the stack to general-purpose registers in the following order: EDI, ESI, EBP, EBX, EDX, ECX, EAX. The SP or ESP is adjusted to reflect the new top position of the stack.

The CALL Instruction: The `call` instruction is used to execute a function. It saves the **return address** (the location where execution should continue after the function finishes) onto the stack and then jumps to the function's address. The function uses the stack to store parameters, local variables, and saved registers.

Answer the questions below

- | Which instruction is used for performing a function call? `call`
- | Which instruction is used to push all registers to the stack? `pusha`

Task 8 - Practice Time

The options on the top bar can be used to:

- Run the program
- Execute the next instruction
- Stop the execution
- Restart the program

Stack: The following instructions demonstrate two instructions, `push` and `pop` used to push data into the stack and pop the data out.

Choose the **Stack** code from the emulator's dropdown and observe the outcome.

Condition	Example	Flags affected by cmp instruction	Flags affected by test instruction
When both values are equal	eax = ebx	Parity Flag, Zero Flag	No flag is impacted
When eax is greater than ebx	eax > ebx	No flag is impacted	Parity Flag, Zero Flag
When eax is less than ebx	eax < ebx	Carry Flag, Sign Flag	Parity Flag, Zero Flag

LEA Instruction: Choose the **Lea Instruction** code from the emulator's dropdown and observe the movement of the data. **Note:** The first instruction is at index 0 in the **Instructions Block**.

While running the MOV instructions, what is the value of [eax] after running the 4th instruction?

(in hex) *0x00000040*

The screenshot shows the Assembly Emulator interface with the following sections:

- Instructions:** A list of 5 instructions:
 0. mov eax,10h
 1. mov ebx,32h
 2. mov ecx,edx
 3. → mov [eax],40h (highlighted)
 4. add [eax],30h
 5. mov [ebx],[eax]
- Registers and Flags:** A list of registers with their values:
 - EAX: 0x00000010
 - EBX: 0x00000032
 - ECX: 0x00000010
 - EDX: 0x00000000
 - ESI: 0x00000000
 - EDI: 0x00000000
 - ESP: 0x00001000
 - EBP: 0x00001000
 - EIP: 0x00000004
- Memory Placeholder:** A table showing memory addresses and their values:

ADDRESS	SIGNED INT	UNSIGNED INT	HEX
0x0	0	0	0x00000000
0x4	0	0	0x00000000
0x8	0	0	0x00000000
0xC	0	0	0x00000000
0x10	64	64	0x00000040
0x4B	0	0	0x00000000
0x4C	0	0	0x00000000
0x50	0	0	0x00000000
- Stack:** A list of stack addresses and their values:
 - 0xF4: 0x00000000
 - 0xF8: 0x00000000
 - 0xFC: 0x00000000
 - 0x00: 0x00000000
 - 0x04: 0x00000000
 - 0x08: 0x00000000
 - 0x0C: 0x00000000
 - 0x10: 0x00000040
 - 0x14: 0x00000000
 - 0x18: 0x00000000
 - 0x1C: 0x00000000

What error is displayed after running the 6th instruction from the MOV instruction section?

Memory to memory data movement is not allowed.

Run the instructions from the stack section. What is the value of eax after the 9th instruction?

(in hex) *0x00000025*

Assembly Emulator

Stack

Instructions

0. mov eax, 10h
1. mov ebx, 15h
2. mov ecx, 20h
3. mov edx, 25h
4. push eax
5. push ebx
6. push ecx
7. push edx
8. → pop eax
9. pop ebx
10. pop ecx
11. pop edx

Registers and Flags

EAX: 0x00000025

EBX: 0x00000015

ECX: 0x00000020

EDX: 0x00000025

ESI: 0x00000000

EDI: 0x00000000

ESP: 0x000000FF4

EBP: 0x00001000

EIP: 0x00000009

Memory Placeholder

ADDRESS	SIGNED INT	UNSIGNED INT	HEX
0x0	0	0	0x00000000
0x4	0	0	0x00000000
0x8	0	0	0x00000000
0xC	0	0	0x00000000
0x10	0	0	0x00000000
0x1B	0	0	0x00000000
0x4C	0	0	0x00000000
0x50	0	0	0x00000000

Stack

Address	HEX
0xFE4	
0xFE8	
0xFEC	
0xFF0	
0xFF4	0x00000020
0xFF8	0x00000015
0xFFC	0x00000010

Run the instructions from the stack section. What is the value of edx after the 12th instruction?

(in hex) *0x00000010*

Assembly Emulator

Stack

Instructions

0. mov eax, 10h
1. mov ebx, 15h
2. mov ecx, 20h
3. mov edx, 25h
4. push eax
5. push ebx
6. push ecx
7. push edx
8. pop eax
9. pop ebx
10. → pop ecx
11. pop edx

Registers and Flags

EAX: 0x00000025

EBX: 0x00000020

ECX: 0x00000015

EDX: 0x00000025

ESI: 0x00000000

EDI: 0x00000000

ESP: 0x000000FFC

EBP: 0x00001000

EIP: 0x0000000B

Memory Placeholder

ADDRESS	SIGNED INT	UNSIGNED INT	HEX
0x0	0	0	0x00000000
0x4	0	0	0x00000000
0x8	0	0	0x00000000
0xC	0	0	0x00000000
0x10	0	0	0x00000000
0x1B	0	0	0x00000000
0x4C	0	0	0x00000000
0x50	0	0	0x00000000

Stack

Address	HEX
0xFE4	
0xFE8	
0xFEC	
0xFF0	
0xFF4	
0xFF8	
0xFFC	0x00000010

Run the instructions from the stack section. After POP ecx, what is the value left at the top of the stack?

(in hex) *0x00000010*

Run the cmp and test instructions. Which flags are triggered after the 3rd instruction?

PF,ZF (Note: Use these abbreviations in alphabetical order with no spaces: CF,PF,SF,ZF)

The screenshot shows an 'Assembly Emulator' window with four main panels:

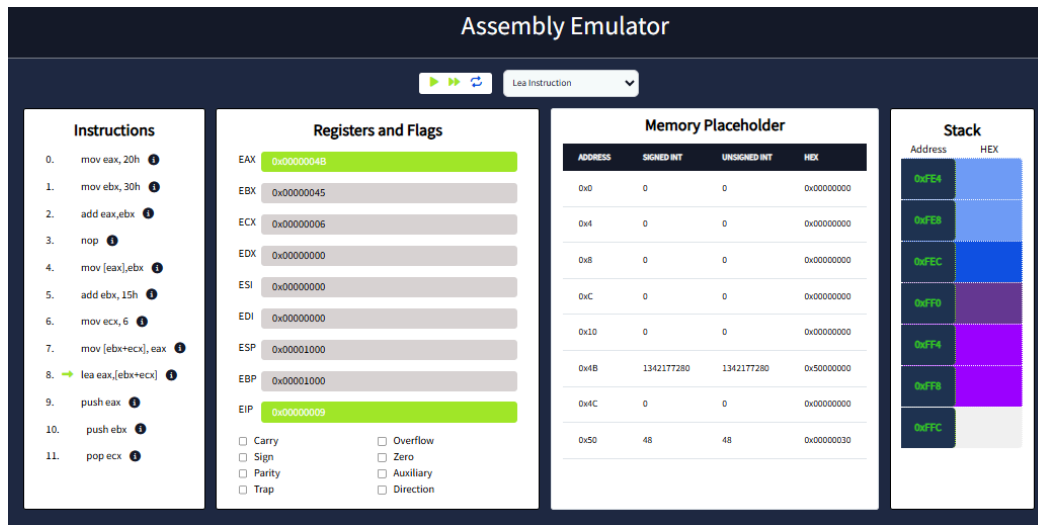
- Instructions:** A list of 11 instructions. The 3rd instruction, 'cmp eax,ebx', is highlighted with a green arrow.
- Registers and Flags:** A table showing the values of registers EAX, EBX, ECX, EDI, ESI, EDI, ESP, and EBP. Below the registers, there are checkboxes for flags: Carry, Sign, Parity, Trap, Overflow, Zero (checked), Auxiliary, and Direction.
- Memory Placeholder:** A table with columns for ADDRESS, SIGNED INT, UNSIGNED INT, and HEX. It shows memory addresses from 0x0 to 0x50.
- Stack:** A table with columns for Address and HEX. It shows memory addresses from 0xFE4 to 0xFFC.

Run the test and the cmp instructions. Which flags are triggered after the 11th instruction?

CF,SF (Note: Use these abbreviations in alphabetical order with no spaces: CF,PF,SF,ZF)

Run the instructions from the lea section. What is the value of eax after running the 9th instruction?

(in hex) *0x0000004B*



Run the instructions from the lea section. What is the final value found in the ECX register?

(in hex) *0x00000045*

Task 9 - Conclusion

We covered the ones that briefly overview how operations are performed in the assembly language.

- Opcodes and how we convert them into the assembly language.
- General instructions such as move, load effective address, shift and rotate.
- Arithmetic instructions such as addition, subtraction, multiplication, and division.
- Conditionals and how we use them for branching.
- Stack, the push and pop instructions, and function calls.